Java testing

Testing from the ground up: the types and the pyramid, unit testing, JUnit, Mockito, integration testing, coverage, and what makes tests trustworthy. The tone is how I would talk each one through with a teammate; the strategy and experience questions are fully spoken.

1. What is software testing, why does it matter, and why automate it?

So software testing is **checking that the code does what it should, and keeps doing it as you change things**. It matters because the alternative is finding bugs in production, where they are expensive and embarrassing, instead of on your machine where they are cheap. **Automated** testing is the real force multiplier: a suite you run on every commit gives you a **safety net to refactor and ship confidently**, catches regressions the moment they appear, and documents how the code is meant to behave, all without someone manually clicking through the app each time. So the benefit is speed and confidence, not just correctness.

2. What types of testing are there, and how do they compare?

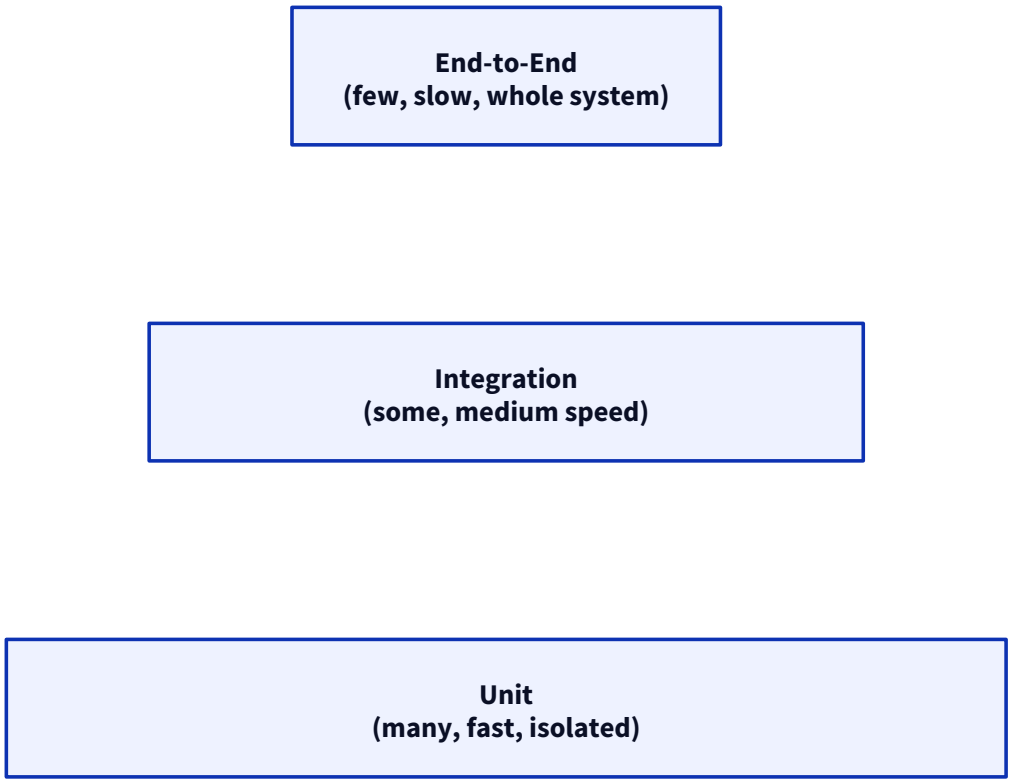
The ones that come up, and the distinctions people ask about:

Comparison	The difference
Unit v/s Integration	Unit tests one class in isolation (deps mocked); integration tests several pieces working together (real DB, real wiring)
Integration v/s End-to-End	Integration tests a slice; E2E drives the whole system like a real user, front to back
Functional v/s Non-functional	Functional checks *what* it does (behaviour); non-functional checks *how well* (performance, security, load)
Smoke v/s Sanity	Smoke is a quick "does it even start / basic paths work"; sanity is a narrow check that a specific fix or area works
Regression v/s Retesting	Regression re-runs tests to catch newly broken things; retesting re-runs the exact case that had the bug, to confirm the fix

Day to day I lean on unit and integration tests heavily, with a thin layer of end-to-end for the critical paths. *(Adapt to your own mix.)*

3. What is the Testing Pyramid?

The Testing Pyramid is the guideline for **how to balance test types**: lots of fast unit tests at the base, fewer integration tests in the middle, and a small number of slow end-to-end tests at the top.



The logic is economic: unit tests are **fast, cheap, and pinpoint the failure**, so you want many; end-to-end tests are **slow, brittle, and vague about what broke**, so you want few. An **inverted pyramid** (mostly E2E) is the anti-pattern, the suite becomes slow and flaky and nobody trusts it. So push most of your coverage down to the unit level.

4. What is unit testing, what makes a good one, and what should (and shouldn't) be tested?

A unit test checks **one small piece of logic in isolation**, one class or method, with its dependencies mocked out. A **good** unit test is **fast, independent, repeatable, and focused on one behaviour**, with a clear name and a single reason to fail. **What to unit test**: your real logic, the branches, the edge cases, the error paths. **What not to**: trivial getters and setters, the framework or the language itself, and anything that is really an integration concern (a real database call). The rule I use: test **behaviour that could plausibly break**, not code that cannot.

5. What is test isolation, and why must tests be independent?

Test isolation means **each test runs on its own, without depending on other tests or shared state**. Tests must be **independent** so that they can run in any order, in parallel, and one failing test does not cascade into others failing for unrelated reasons. The classic violation is **shared mutable state** (a static field or a database row one test sets and another reads), which makes tests pass or fail depending on order, the start of flakiness. So each test sets up its own world and tears it down.

6. How do you test private methods and exception scenarios?

On **private methods**: you generally **do not test them directly**. A private method is an implementation detail, so you test it **through the public method that uses it**, if the private logic matters, some public behaviour exercises it. If a private method is so complex it feels like it needs its own test, that is usually a sign it should be **extracted into its own class**. On **exceptions**: you assert that the code **throws the expected exception for bad input**, in JUnit 5 with `assertThrows(SomeException.class, () -> code())`, and you can then assert on the message or type. So you test the failure path as deliberately as the happy path.

7. What is JUnit, and what changed across versions?

JUnit is **the standard testing framework for Java**, it runs your test methods and reports pass/fail. On versions: **JUnit 4** was the long-time standard (single jar, `@RunWith`); **JUnit 5** was a rewrite into three parts, the **Platform** (the engine that launches tests), **Jupiter** (the new `@Test` API and annotations), and **Vintage** (runs old JUnit 4 tests), plus a much richer feature set. **JUnit 6** is the recent step (2025), which mainly modernises the baseline (requires a newer Java, drops deprecated bits) while keeping the JUnit 5 programming model, so it is more cleanup than reinvention. I would confirm the exact JUnit 6 changes against the release notes, since it is new.

8. What are the common JUnit annotations and the test lifecycle?

The ones you use constantly:

Annotation	What it does
`@Test`	Marks a test method
`@BeforeEach` / `@AfterEach`	Runs before/after **each** test (fresh setup per test)
`@BeforeAll` / `@AfterAll`	Runs once before/after **all** tests in the class (static)
`@DisplayName`	A readable name for the test in reports
`@Nested`	Groups related tests in an inner class
`@Tag`	Labels tests so you can run a subset (e.g. "slow")
`@Disabled`	Skips a test (with a reason)

The **lifecycle**: `@BeforeAll` once, then for each test `@BeforeEach` → the `@Test` → `@AfterEach`, and finally `@AfterAll` once. The `Each` v/s `All` split is the thing to remember: per-test setup keeps tests isolated, all-once setup is for expensive shared resources.

9. What assertions and test styles does JUnit 5 offer?

Beyond the basic `assertEquals` / `assertTrue`, the useful ones: **`assertThrows`** checks that a block throws the expected exception; **`assertAll`** groups several assertions so you see **all** the failures at once instead of stopping at the first; **parameterized tests** (`@ParameterizedTest` with `@ValueSource` / `@CsvSource`) run the same test over many inputs, which kills duplication; and **dynamic tests** (`@TestFactory`) generate tests at runtime when the cases are not known upfront. Parameterized tests are the one I reach for most, one test body, a table of inputs and expected outputs.

10. What is Mockito, why mock, and how do Mock, Stub, and Spy differ?

Mockito is **the mocking framework** you use to replace a class's real dependencies with fakes in a unit test. You **mock** so you can test a class **in isolation**, without spinning up its real database or network calls, and so you can control what the dependency returns and check how it was used. The three terms people muddle:

- **Stub**: returns **canned answers** (you set it up to return a value); it is about **state**.
- **Mock**: a fake you also **verify interactions on** (was this method called, with what); it is about **behaviour**.
- **Spy**: wraps a **real object**, so real methods run unless you stub them, useful for partial mocking.

The short version: a stub feeds data in, a mock checks the calls, a spy is a real object you can selectively override.

11. What do @Mock, @Spy, @InjectMocks, and @Captor do?

These are the Mockito annotations that wire a test up:

Annotation	Purpose
`@Mock`	Creates a mock of a dependency
`@Spy`	Creates a spy (real object, selectively stubbed)
`@InjectMocks`	Creates the object under test and **injects the mocks/spies into it**
`@Captor`	Creates an `ArgumentCaptor` to capture arguments passed to a mock

The usual shape: `@InjectMocks` on the service you are testing, `@Mock` on each of its dependencies, and Mockito wires them together.

12. when() v/s doReturn(), verify(), and ArgumentCaptor.

`when(x).thenReturn(v)` is the normal way to stub a return value, but it **actually calls the method during setup**, which is a problem for **spies and void methods**. **`doReturn(v).when(x)`** does the same thing **without calling the real method**, so it is the safe form for spies and voids. **`verify(mock).method(args)`** checks that a method **was called** (and how many times), which is how you test interactions. **`ArgumentCaptor`** grabs the actual argument a mock was called with, so you can assert on it, useful when the interesting thing is *what* got passed, not just that the call happened.

13. How do you mock static methods, final classes, and constructors, and what are the pitfalls?

Modern Mockito handles the things that used to need PowerMock. **Static methods**: `mockStatic(SomeClass.class)` in a try-with-resources scope (Mockito 3.4+). **Final classes and methods**: supported since Mockito 2 with the inline mock maker (no extra library now). **Constructors**: `mockConstruction(...)` intercepts `new` calls (Mockito 3.5+). The **pitfalls**: **over-mocking** (mocking so much you are testing the mocks, not the code), mocking **types you do not own** (better to wrap them), forgetting that **static/constructor mocks are scoped** (only active inside the block), and stubbing calls that never happen. The general smell: if you are reaching for static mocking a lot, the design is probably too tightly coupled.

14. What is integration testing, when do you write it, and what are the Spring Boot slices?

Integration testing checks that **several parts work together**, the service, the real repository, the actual database, the wiring, rather than one class in isolation. You write it **when the risk lives in the seams**: the SQL actually runs, the JSON actually serializes, the Spring context actually wires up. Spring Boot gives you **test slices** so you do not boot the whole app every time:

- `@SpringBootTest` : loads the **full** application context (broadest, slowest).
- `@WebMvcTest` : just the **web layer** (controllers + `MockMvc`), for testing endpoints.
- `@DataJpaTest` : just the **JPA/repository layer**, with a database.
- `@JdbcTest` : just the **JDBC** bits.

The slices keep integration tests focused and faster by only loading the part you are testing.

15. Embedded databases v/s Testcontainers, and why are Testcontainers preferred?

An **embedded/in-memory database** (like H2) is fast and needs no setup, but it is **not your real database**, so its SQL dialect and behaviour differ from Postgres or MySQL, and tests can pass against H2 then fail in production. **Testcontainers** spins up the **real database in a Docker container** for the test, so you test against the actual engine you ship on. That is why Testcontainers is **preferred**: it catches the dialect and behaviour differences an in-memory DB hides, at the cost of needing Docker and being a bit slower. On **test data**, the key is that each test sets up and cleans its own data (or runs in a rolled-back transaction) so tests stay independent.

16. What is code coverage, what are its types, and can high coverage still hide bugs?

Code coverage measures **how much of your code the tests actually execute**. The types, roughly increasing in strictness: **statement** (was each line run), **branch** (was each `if` / `else` path taken), **condition** (was each boolean sub-expression evaluated both ways), and **path** (were the combinations of branches covered). Is **100% necessary**? No, chasing it wastes effort on trivial code and gives false confidence. And yes, **high coverage can absolutely hide bugs**, because coverage only proves the code **ran**, not that you **asserted the right thing**. A test with no meaningful assertions can execute a method fully and catch nothing. So I treat coverage as a **signal for what is untested**, not a quality score.

17. What makes a test maintainable v/s brittle?

A **maintainable** test is **readable** (you can tell what it checks at a glance), tests **behaviour not implementation**, has **no duplication** (shared setup extracted), and is **fast and independent**. A **brittle** test breaks on changes that did not actually break the behaviour, usually because it is **coupled to implementation details** (asserting exact internal calls, over-mocking) or has **fragile assertions** (asserting a whole object when you care about one field). The mindset: test **what the code does**, not **how it does it**, so a refactor that preserves behaviour keeps the tests green.

18. What are flaky tests, and how do you fix them?

A flaky test is one that **passes and fails without the code changing**, and they are poison because they train people to ignore failures. The usual causes: **timing** (a `Thread.sleep` that is sometimes too short), **shared state** between tests, **test-order dependence**, **real external services**, and **randomness or real clocks**. To fix them: **remove** `Thread.sleep` and wait on a real condition (a polling/await helper), **isolate state** so tests do not leak into each other, **mock external dependencies**, and **control time and randomness** (inject a fixed clock/seed). The goal is **determinism**, same input, same result, every run.

19. How do you test controllers, services, and repositories?

Each layer gets a different treatment. **Controllers**: with `@WebMvcTest` and `MockMvc` (or `WebTestClient`), you fire HTTP requests and assert on the status and JSON, mocking the service beneath. **Services**: **plain unit tests** with Mockito, mock the repositories and collaborators and test the business logic and its branches directly, this is where most of your tests live. **Repositories**: with `@DataJpaTest` against a real database (Testcontainers), because the thing you are testing is the actual query, which a mock cannot verify. So: mock below the controller, unit-test the service, and hit a real DB for the repository.

20. What is your testing strategy, which frameworks do you use, and how have you improved test quality?

My default strategy follows the **pyramid**: a broad base of fast **unit tests** (JUnit 5 + Mockito) on the business logic, a focused layer of **integration tests** with **Testcontainers** for the repository and API seams, and a thin set of **end-to-end** tests on the critical flows. The frameworks I reach for are **JUnit 5, Mockito, and Testcontainers**, plus Spring Boot's test slices. On **improving quality**, the moves that paid off were **replacing in-memory H2 with Testcontainers** (which caught dialect bugs H2 was hiding), **killing flaky tests** by removing `Thread.sleep` and shared state, and shifting tests from **implementation to behaviour** so refactors stopped breaking them. *(Adapt to your own project.)*

21. If unit tests pass, can production still fail? And tell me about a bug better testing could have caught.

Yes, absolutely, and it is important to be honest about why. **Unit tests mock the dependencies**, so they never exercise the real database, the real network, config, concurrency, or the wiring between services, so integration issues, a bad SQL dialect, a serialization mismatch, or a race condition all sail past a green unit suite. That is exactly the kind of bug I have seen bite: something that **only shows up when the real pieces talk to each other**, which a unit test with mocks could never have caught, and which an **integration test with Testcontainers** would have. The lesson I keep is to put tests at the level where the risk actually lives, not just where they are easy to write. *(Adapt to a real bug you have seen.)*

22. How do you review unit tests in code review, and what standards does your team follow?

When I review a test I am asking: **does it test behaviour or implementation** (implementation-coupled tests are a red flag), **would it actually fail if the code broke** (assertions that assert nothing are common), **is it readable and independent**, and **is it over-mocked**. The standards I would hold a team to: **test behaviour not internals**, **one clear reason to fail per test**, **no shared mutable state or order dependence**, **no `Thread.sleep`**, **fix or delete flaky tests immediately**, and **coverage as a signal, not a target**. Most of this is catchable in review, and a good test is one a teammate can read and understand what it protects. *(Adapt to your own team.)*

23. How would you design a testing strategy for a large microservices platform, and balance the test types?

Across many services I would push the balance toward the **pyramid within each service** plus **contract tests between them**. Each service owns a strong base of **unit tests** and **integration tests** (Testcontainers for its own DB and APIs). Between services, I would rely on **contract tests** (consumer-driven, like Pact) rather than sprawling end-to-end suites, because full E2E across many services is slow, flaky, and hard to own, so contract tests let each service verify it honours its API without booting the whole platform. I would keep **end-to-end** tests to a **thin set of the most critical user journeys**. So the balance is: most tests unit, solid integration per

service, contract tests at the boundaries, and E2E only where a whole-system failure would really hurt. *(Adapt to your platform.)*

24. Common mistakes.

- **Over-mocking dependencies**, until you are testing the mocks instead of the code.
- **Testing implementation instead of behaviour**, so every refactor breaks the tests.
- **Using real external services in unit tests**, which makes them slow and flaky.
- **Sharing mutable state across tests**, the root of order-dependence and flakiness.
- **Ignoring edge cases** and only testing the happy path.
- **Depending on test execution order**, which breaks the moment tests run in parallel.
- **Writing fragile assertions** (asserting a whole object when one field matters).
- **Using `Thread.sleep()` in tests** instead of waiting on a real condition.
- **Ignoring flaky tests** instead of fixing or deleting them.
- **Chasing 100% coverage** as a goal rather than testing what matters.